



计算思维与实践

哈尔滨工业大学（深圳）

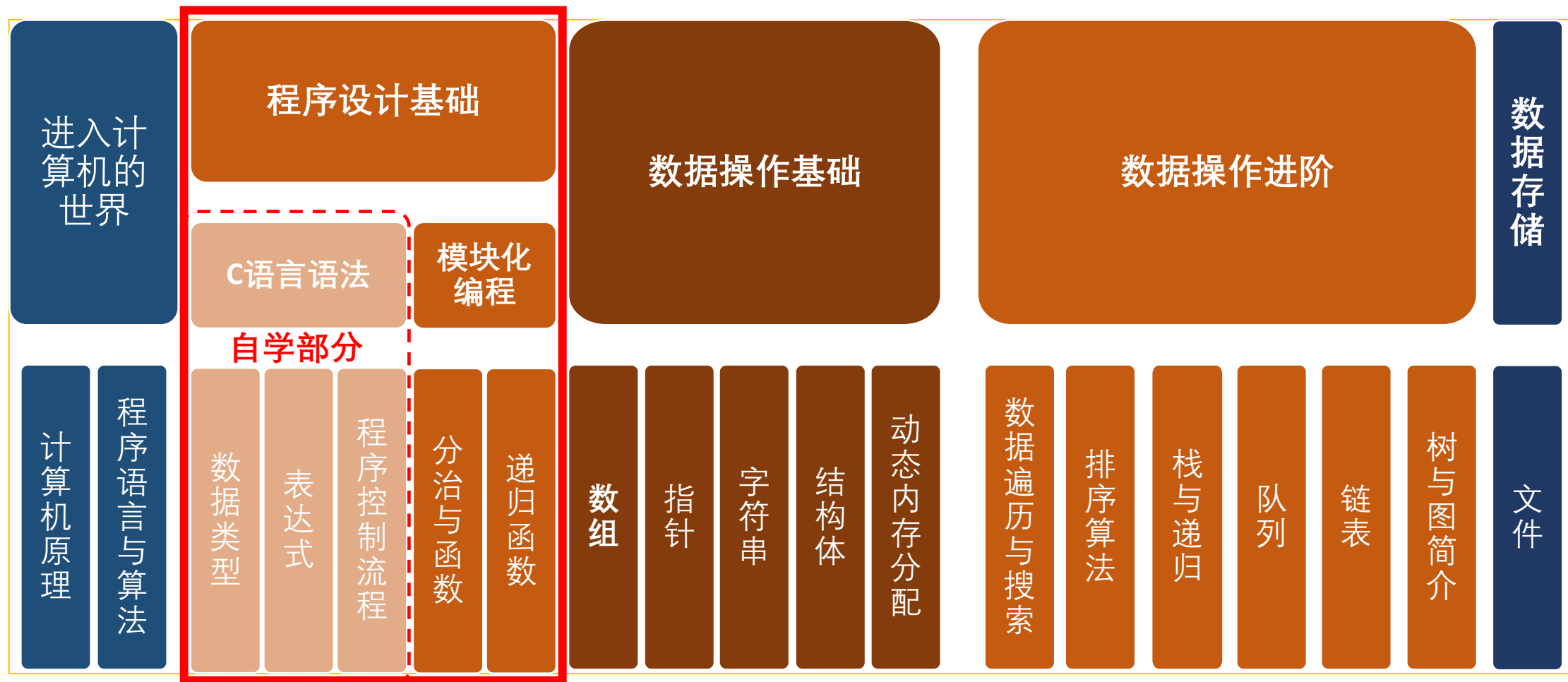
计算机科学与技术学院

大数据技术中心

张保权

课程内容安排

2



程序设计思想与数据操作方法融会贯通，内容由浅入深

C语言什么样？

3

简单的C程序：“Hello world!”

```
#include <stdio.h>
main()
{
    printf("Hello world!\n");
}
```

复杂点：加法计算器

```
#include <stdio.h>

int Add(int a, int b)
{
    return (a + b);
}

main()
{
    int x, y, sum = 0;

    printf("Input two integers:");
    scanf("%d%d", &x, &y);
    sum = Add(x, y);
    printf("sum = %d\n", sum);
}
```

C语言什么样？

4

■ 第一个程序范例——打印 "Hello World!"

```
#include <stdio.h>
main()
{
    printf("Hello world!\n");
}
```

以#开头，编译预处理指令

头文件

Every C program must have a main function
The execution of C program starts from **main()** function

■ 考考你：

- 如何把 "Hello" 和 "world!" 分别打印在两行？

C语言程序设计要学什么？

5

- 要学会和计算机进行沟通交流时要遵从的语法
 - C语言有C的语法、Java语言有Java语言的语法, 等等
- 要理解每种语法后面的语义
 - 虽然每种语言的语法各有不同，但很多语言都有相同或相近的语义，这种语义，往往是我们想要传达给计算机意图
- 要建立起从计算机的角度来思考问题
 - 要能够在大脑中直接用计算机的语言来进行思考
 - 这实际上是思维方式的一种改变

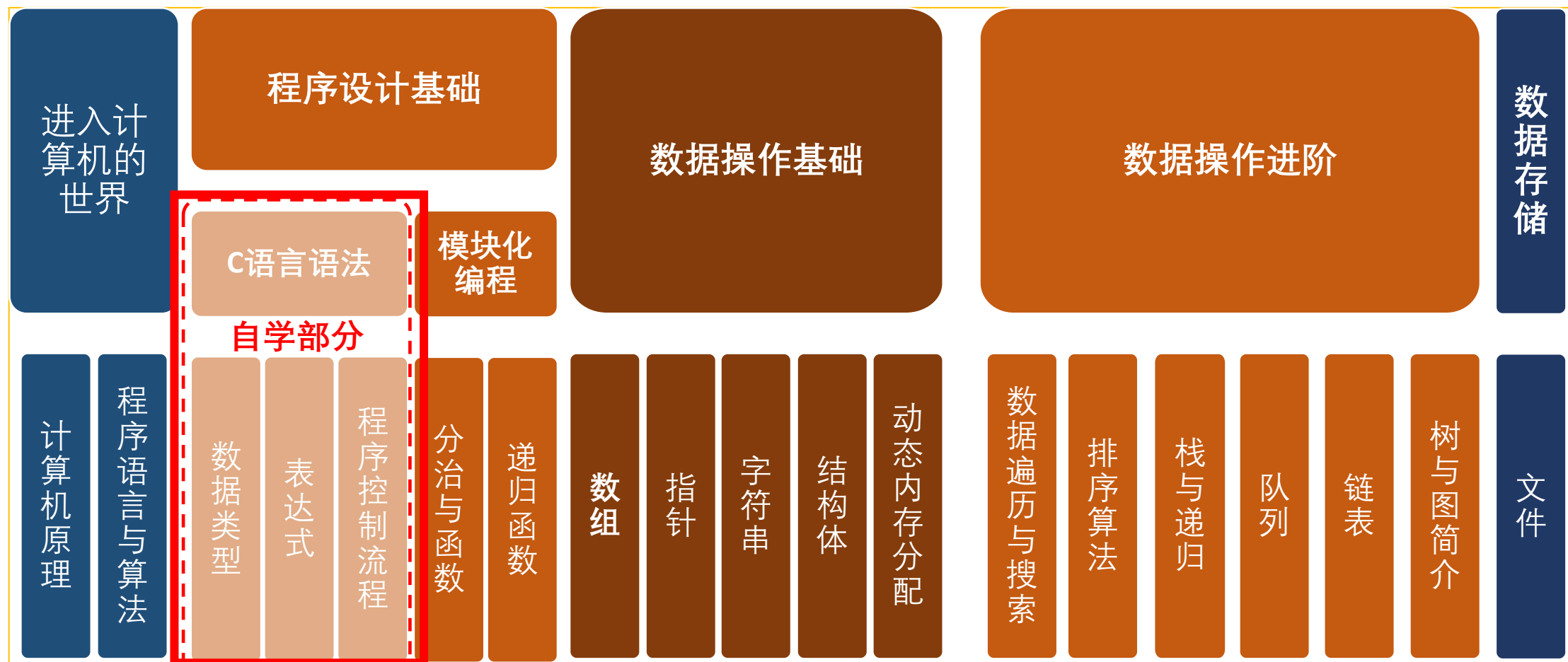
语言学习Learning与习得Acquiring

6

- 语言学习? **No!**
- 只有把语言内化成自己的内在的思维模式，我们才叫习得了一门语言，语言习得是能力的培养而不是技能的学习。
- 语言习得的四个层次
 - **词汇**：高级语言往往只有有限的关键词和词语命名规范（靠自己背诵）
 - **语法、句法**：不同的语言结构对应不同的语法（靠自己记忆、掌握）
 - **语义**：进入计算机世界的桥梁，理解每个词语、语法背后对应的计算机系统的变化，是我们课程讲授的重点
 - **语用**：将语言能力从**流畅交流**升华到**职业作家**！

课程内容安排

7



程序设计思想与数据操作方法融会贯通，内容由浅入深

第三讲 学习内容—数据类型、表达式、程序控制流程

8

- 3.1 数据类型
 - 3.1.1 数据与变量
 - 3.1.2 基本数据类型
 - 3.1.3 复合与抽象类型
- 3.2 表达式与运算
 - 3.2.1 算术运算与表达式
 - 3.2.2 类型转换
 - 3.2.3 常用数学函数
- 3.3 程序控制流程
 - 3.3.1 顺序与选择结构
 - 3.3.2 循环结构
 - 3.3.3 流程控制语句

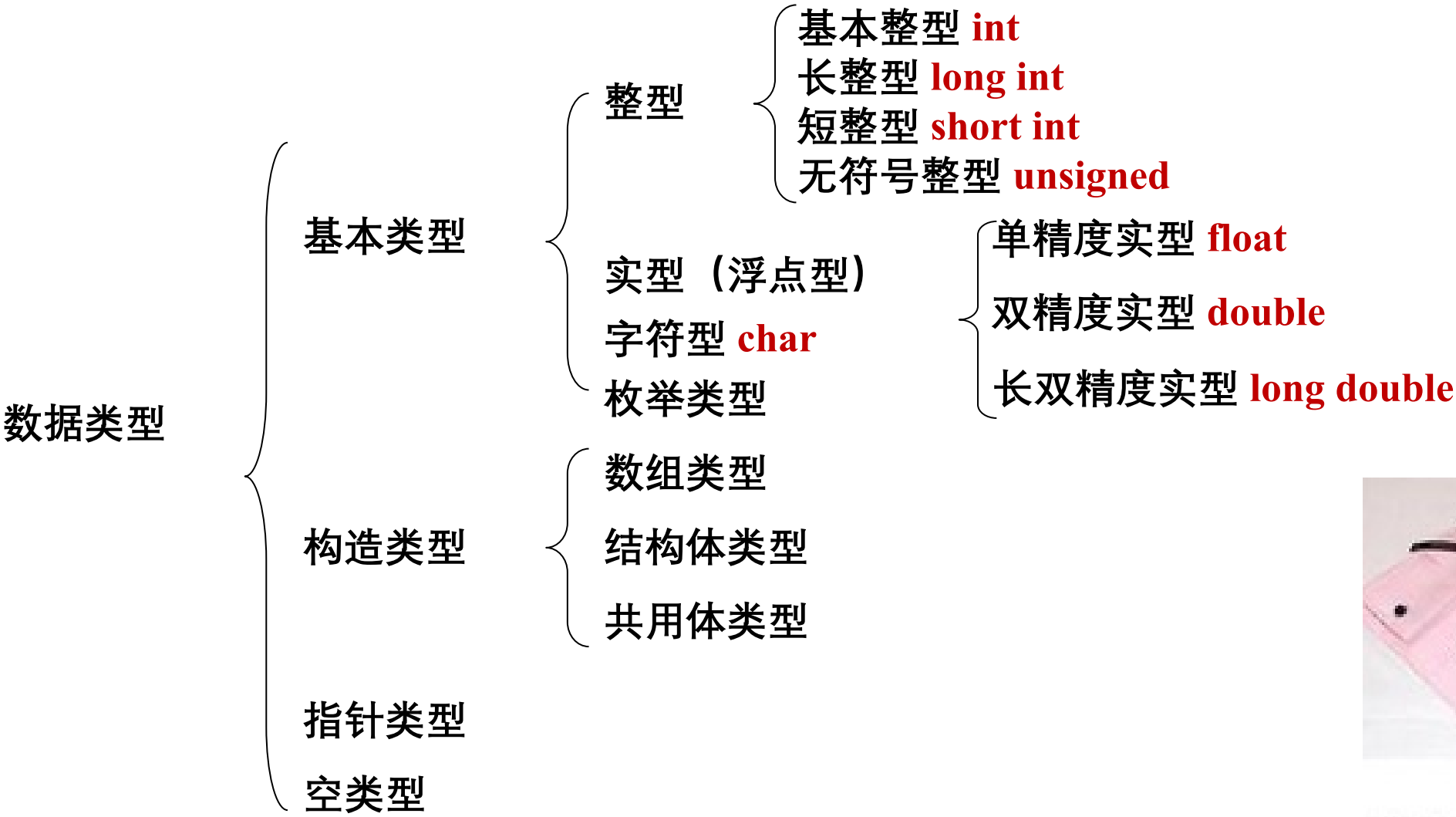


3.1 数据类型-在C语言中为什么要引入数据类型？

- 对计算机系统和硬件本身而言，数据类型（Data Type）的概念其实是不存在的
 - 冯·诺依曼体系结构中，程序代码和数据以二进制存储
- 引入数据类型的主要目的
 - 有效地组织数据，把数据分成所需内存大小不同的数据
 - 规范数据的使用
 - 提高程序的可读性
 - 方便用户的使用

3.1 数据类型 - C语言中有哪些数据类型?

10



3.1.1 数据与变量- C语言程序中有哪些数据形式?

11

- C语言程序中的数据一共有两种形式
 - 常量 (Constant)
 - 在程序中不能改变其值的量
 - 变量 (Variable)
 - 其值在程序执行过程中是可以改变的



3.1.1 数据与变量- 如何定义一个常量? ——宏定义与const常量

12

- 宏常量 (Macro Constant)

- 也称符号常量

- 宏定义

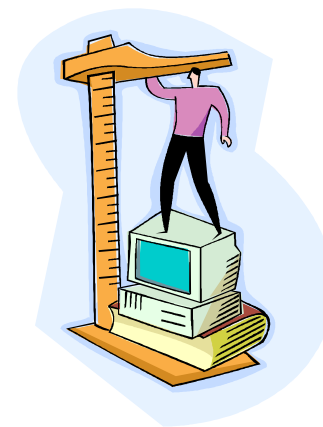
- `#define` 标识符 字符串
- 标识符——宏名 (Macro Name)
- 一般采用全大写字母表示

- `const`常量

- `Const` 数据类型 常量名 = 常量值;
- `const`常量在使用过程中不可修改
- `const`声明常量要在一个语句内完成

`#define` 与 `const` 区别

通常情况下, 建议使用 `const` 关键字来定义常量, 因为它具有类型检查和作用域的优势, 而 `#define` 仅进行简单的文本替换, 可能会导致一些意外的问题。



3.1.1 数据与变量- 如何声明变量的类型?

13

- 变量的值在程序执行过程中是可以改变的
- 变量的声明(Variable Declaration)
 - 类型关键字 变量名;
- 使用变量的基本原则
 - 变量必须先声明，后使用
- 一条声明语句可声明若干个同类型的变量
 - `int a, b, c;`
- 声明的顺序无关紧要

```
1  main()  
2  {  
3      int    a;  
4      float  b;  
5      char   c;  
6      a = 1;  
7      b = 2.5;  
8      c = 'A';  
9  }
```

3.1.1 数据与变量- 如何给变量赋值?

14

- 声明变量的同时为变量赋值(Assignment)—变量的初始化 (Initialize)
- 未被初始化的变量的值会是什么?

- 其值为随机数 (乱码)

- 赋值表达式语句:

变量 = 表达式 ;

- 规则:

变量 ← 表达式 左值和右值类型一致

- 多重赋值 (Multiple Assignment)

变量1 = 变量2 = 表达式 ;



3.1.1 数据与变量-如何给变量赋值?

15

Example:

→ `int a, b;`

→ `float x, y;`

`. . .`

→ `a = b = 0;`

→ `x = y = 100.0;`

a 0

b 0

x 100.0

y 100.0

第三讲 学习内容—数据类型、表达式、程序控制流程

16

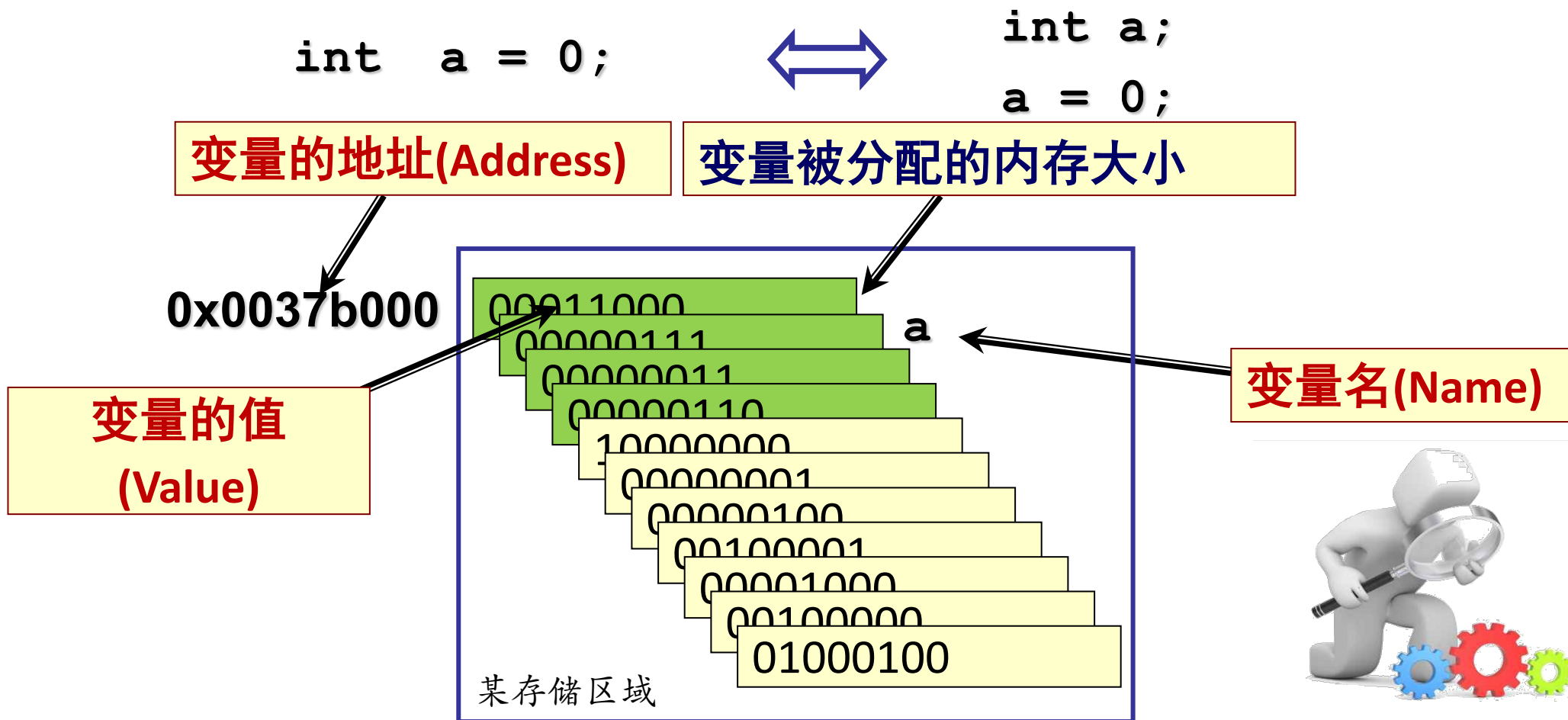
- 3.1 数据类型
 - 3.1.1 数据与变量
 - 3.1.2 基本数据类型
 - 3.1.3 复合与抽象类型
- 3.2 表达式与运算
 - 3.2.1 算术运算与表达式
 - 3.2.2 类型转换
 - 3.2.3 常用数学函数
- 3.3 程序控制流程
 - 3.3.1 顺序与选择结构
 - 3.3.2 循环结构
 - 3.3.3 流程控制语句



3.1.2 基本数据类型- 变量有哪些属性?

17

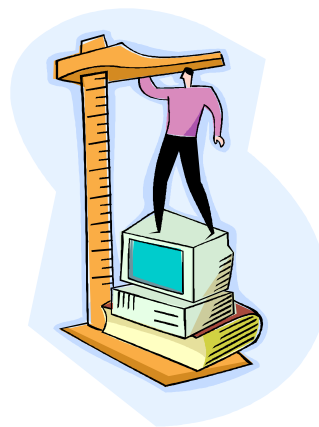
- 变量的类型(Type)—决定变量被分配内存空间的大小



3.1.2 基本数据类型- 不同类型数据占用的内存

18

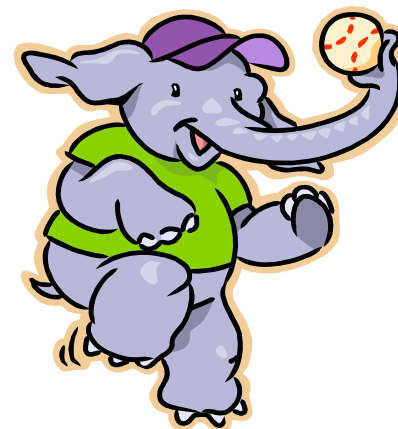
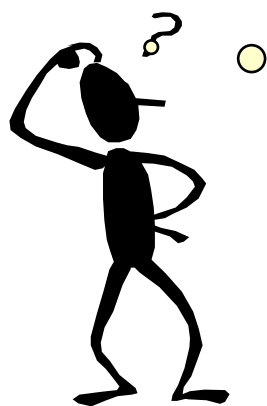
- `int`—基本整型，C标准未规定，系统相关
 - 在目前大多数系统上占4个字节
- `short int`，简写为`short`
 - 短整型，2个字节
- `long int`，简写为`long`
 - 长整型，4个字节
- `unsigned`—无符号整型（正整数和0）
 - 用来修饰`int`、`short`和`long`
- `float`
 - 单精度实型，4个字节
- `double`
 - 双精度实型，8个字节
- `long double`
 - 长双精度实型，系统相关
 - VC++中占8个字节
- `char`
 - 字符型，1个字节



3.1.2 基本数据类型- 不同类型数据的取值范围

19

- 当向变量赋超出其表数范围的数值时，结果会怎样呢？
 - Assign a short with larger number?
 - 产生数值溢出（Overflow），得到奇怪的结果



3.1.2 基本数据类型- C程序中的常见符号

20

- **关键字 (Keyword)**
 - 又称保留字 (Reserved Word)
- **标识符 (Identifier)**
 - 系统预定义标识符
 - 用户自定义标识符
 - 变量名, 函数名,
- **运算符 (Operator)**
 - 详见附录C
- **分隔符 (Separator)**
 - 空格、回车/换行、逗号等
- **其他符号**
 - {和}标识函数体或语句块
 - /*和*/或//是程序注释的定界符
- **赋值符号**

第三讲 学习内容—数据类型、表达式、程序控制流程

21

- 3.1 数据类型
 - 3.1.1 数据与变量
 - 3.1.2 基本数据类型
 - 3.1.3 复合与抽象类型
- 3.2 表达式与运算
 - 3.2.1 算术运算与表达式
 - 3.2.2 类型转换
 - 3.2.3 常用数学函数
- 3.3 程序控制流程
 - 3.3.1 顺序与选择结构
 - 3.3.2 循环结构
 - 3.3.3 流程控制语句



3.1.3 复合与抽象类型- 什么是构造数据类型?

22

- 构造数据类型是一种自定义的数据类型，允许我们组合不同基本类型的变量成为一个单一的实体。
- 典型代表的数据类型：
 - 数组
 - 枚举
 - 共用体
 - 结构体



3.1.3 复合与抽象类型-什么是抽象数据类型?

23

- 有没有一种类型能定义车呢?
 - 除车的属性，还包括对车的操作（启动、加速、减速、刹车等）
- **抽象数据类型（Abstract Data Type, ADT）**
 - 不单纯是数据的集合，增加了对数据的操作
 - 类型的表示和操作的实现细节对外是不可见的
 - 能达到更好的信息隐藏效果

第三讲 学习内容—数据类型、表达式、程序控制流程

24

- 3.1 数据类型
 - 3.1.1 数据与变量
 - 3.1.2 基本数据类型
 - 3.1.3 复合与抽象类型
- 3.2 表达式与运算
 - 3.2.1 算术运算与表达式
 - 3.2.2 类型转换
 - 3.2.3 常用数学函数
- 3.3 程序控制流程
 - 3.3.1 顺序与选择结构
 - 3.3.2 循环结构
 - 3.3.3 流程控制语句



3.2.1 算术运算符和表达式

25

- 何谓运算符(Operator)和操作数(Operand) ?

Example:

W + Z

操作数

运算符

操作数



3.2.1 算术运算符和表达式—除法(Division)

26

Example:

W / Z

$$11 / 5 = 2$$

整数除法

(Integer Division)

■ W and Z are integers

$$11.0 / 5 = 2.2$$

浮点数除法

(Floating Division)

■ W or Z or both are floats

3.2.1 算术运算符和表达式

27

运算符	含义	操作数个数	优先级	结合性
-	取相反数 (Opposite number)	1个 (一元)	最高	从右向左
* / %	乘法 (Multiplication) 除法 (Division) 求余 (Modulus)	2个 (二元)	较低	从左向右
+ -	加法 (Addition) 减法 (Subtraction)	2个 (二元)	最低	从左向右

3.2.1 算术运算与表达式—复合的赋值运算符

28

- 三种赋值形式:

- 1.简单赋值 (Simple Assignment)

变量 = 表达式 ;

- 2.多重赋值 (Multiple Assignment)

变量1 = 变量2 = 表达式 ;

- 赋值运算的结合性是右结合

- 3.复合的赋值 (Combined Assignment)

变量x 运算符 op = 表达式 ;

变量x = 变量x 运算符 op 表达式 ;

3.2.1 算术运算与表达式—复合的赋值运算符

29

- 变量x 运算符op = 表达式；

注：op= 中间没有空格

- 简写（Shorthand）形式更直观，且执行效率也更高一些

Operation	Examples of expression	Description
+=	num += 5;	num = num + 5;
-=	num -= 5;	num = num - 5;
*=	num *= 5;	num = num * 5;
/=	num /= 5;	num = num / 5;
%=	num %= 5;	num = num % 5;

3.2.1 算术运算与表达式—复合的赋值运算符

30

已知 `int a = 3;`
执行 `a += a -= a * a` 后，变量 `a` 的值？

`a += a -= a * a`

`a += a -= 9`

`a += -6`

`a = -12`

-12

3.2.1 算术运算与表达式—复合的赋值运算符

31

执行 $a += a -= a *= a$ 后，变量 a 的值？

$a += a -= a *= a$

$a += a -= 9$

$a += 0$

$a = 0$

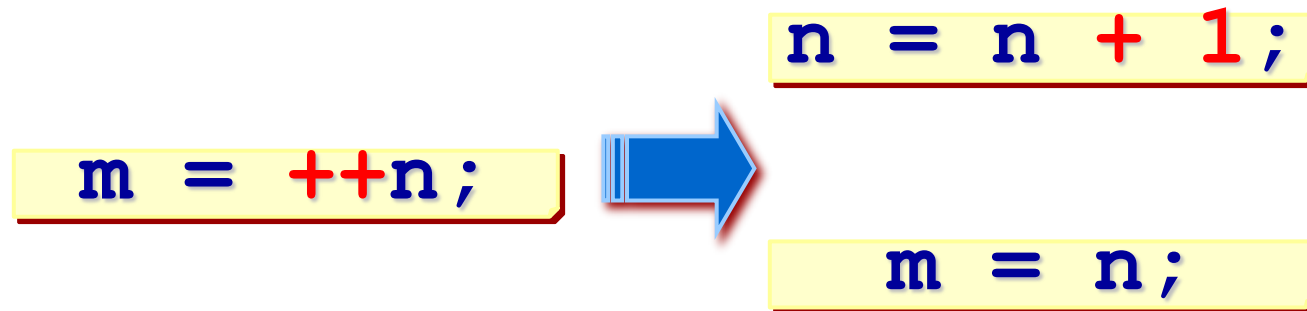
0



3.2.1 算术运算与表达式—前缀的增1和减1运算符

32

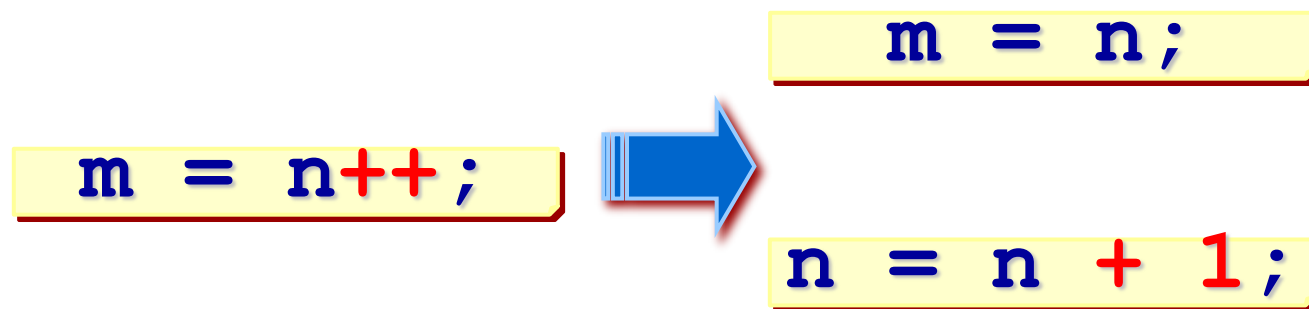
- 增1和减1运算符(Increment and Decrement)
- $n++$, $n--$, $++n$, $--n$
 - $++$ 让参与运算的变量加1, $--$ 让参与运算的变量减1
 - 作为前缀(prefix)运算符时, 先加/减1, 然后取 n 的值



3.2.1 算术运算与表达式—后缀的增1和减1运算符

33

- 增1和减1运算符(Increment and Decrement)
- $n++$, $n--$, $++n$, $--n$
 - $++$ 让参与运算的变量加1, $--$ 让参与运算的变量减1
 - 作为**后缀(postfix)**运算符时, 先取 n 的值, 然后加/减1



- 良好的程序设计风格提倡: 在一行语句中, 一个变量只出现一次增1或减1运算

3.2.1 算术运算与表达式—前缀的增1运算符示例

34

Prefix Example:

$j = ++i - 2;$

similar to

→ $i = i + 1;$

→ $j = i - 2;$

i 6

j 4

3.2.1 算术运算与表达式—后缀的增1运算符示例

35

Postfix Example:

$j = i++ - 2;$

similar to

→ $j = i - 2;$

→ $i = i + 1;$

i 6

j 3

第三讲 学习内容—数据类型、表达式、程序控制流程

36

- 3.1 数据类型
 - 3.1.1 数据与变量
 - 3.1.2 基本数据类型
 - 3.1.3 复合与抽象类型
- 3.2 表达式与运算
 - 3.2.1 算术运算与表达式
 - 3.2.2 类型转换
 - 3.2.3 常用数学函数
- 3.3 程序控制流程
 - 3.3.1 顺序与选择结构
 - 3.3.2 循环结构
 - 3.3.3 流程控制语句



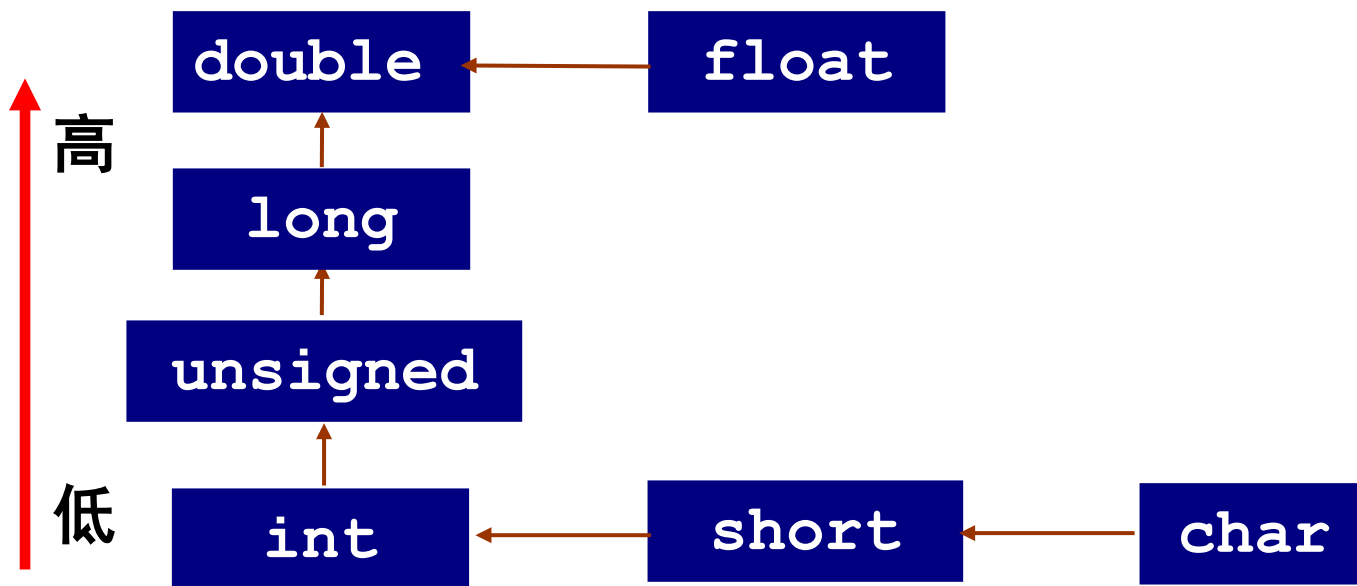
3.2.2 类型转换—类型提升及其规则

37

- 算术表达式中
 - 问题：不同类型数据的运算结果的类型是什么？
 - 取值范围较大的那种类型
 - C编译器将所有操作数都转换成占内存字节数最大的操作数的类型，称为类型提升（Type Promotion）

- 类型提升规则

为什么这样设计？



3.2.2 类型转换—类型强转

38

- 把表达式的值转为任意类型——类型强转 (Casting)
(类型) 表达式

Example:

```
int    x = 10;
```

```
float  y;
```

```
y = (float)x;
```

→ (float)10

→ 10.000000

不改变x

x

10

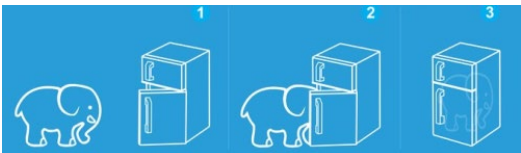
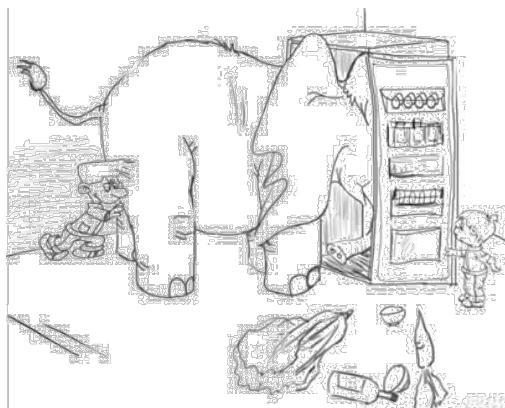
y

10.000000

3.2.2 类型转换—类型强转的安全问题

39

- 问题：在不同类型数据间赋值，是安全的吗？
 - 取值范围小的类型赋值给取值范围大的类型是安全的，反之是不安全的
 - 数值溢出(Overflow)
 - 把大象放到冰箱里



数据类型	所占字节数 (bytes)	取值范围
char	1	-128~127
signed char	1	-128~127
unsigned char	1	0~255
short int	2	-32768~32767
signed short int	2	-32768~32767
unsigned short int	2	0~65535
int	4	-2147483648~2147483647
signed int	4	-2147483648~2147483647
unsigned int	4	0~4294967295
unsigned long int	4	0~4294967295
long int	4	-2147483648~2147483647
signed long int	4	-2147483648~2147483647
float	4	$3.4 \times 10^{-38} \sim 3.4 \times 10^{38}$
double	8	$-1.7 \times 10^{-308} \sim 1.7 \times 10^{308}$
long double	10	$-1.2 \times 10^{4932} \sim 1.2 \times 10^{4932}$

3.2.2 类型转换—类型强转的安全问题

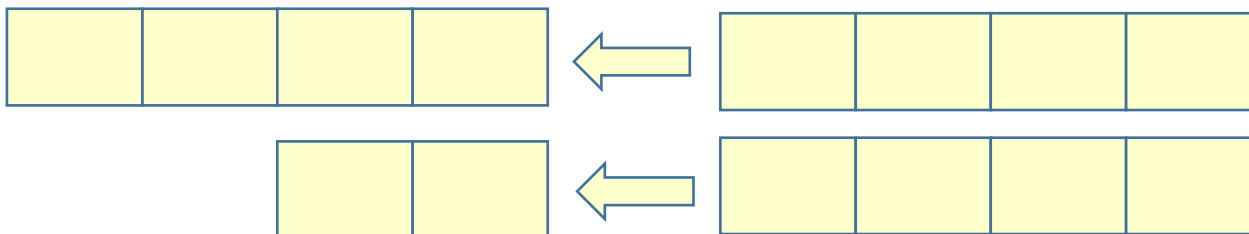
40

- 问题：从高精度向低精度转换时，损失什么信息？
 - 因低精度的数据位数比高精度的少，容纳不下高精度的所有信息，就会丢失信息，出现舍入（Round），也称截断（Truncation）



```
warning: '=': conversion from 'float' to 'int', possible loss of data
```

- 浮点数转为整数，会丢失小数部分，某些情况下整数部分精度也会损失



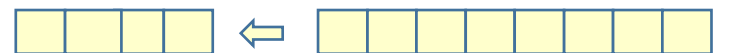
3.2.2 类型转换—类型强转的安全问题

41

- 问题：双精度浮点数转为单精度，结果会怎样？
 - 可能因有效数字位数不够而出现精度损失
 - 尾数所占位数决定实数的精度，不同系统下实数精度不同
 - Visual C++中，`float`类型的有效位只有6~7位，`double`类型的有效位只有15~16位，有效位后的数字不精确

```
#include <stdio.h>
main()
{
    float    a = 123456.789e4;
    double   b = 123456.789e4;
    printf("%f\n%f\n", a, b);
}
```

double常量



1234567936.000000+
1234567890.000000+

warning: 'initializing': truncation from 'const double' to 'float'

第三讲 学习内容—数据类型、表达式、程序控制流程

42

- 3.1 数据类型
 - 3.1.1 数据与变量
 - 3.1.2 基本数据类型
 - 3.1.3 复合与抽象类型
- 3.2 表达式与运算
 - 3.2.1 算术运算与表达式
 - 3.2.2 类型转换
 - 3.2.3 常用数学函数
- 3.3 程序控制流程
 - 3.3.1 顺序与选择结构
 - 3.3.2 循环结构
 - 3.3.3 流程控制语句



3.2.3 常用的标准数学函数

43

- `#include <math.h>`

函 数 名	功 能
<code>sqrt(x)</code>	计算 x 的平方根, x 应大于等于 0
<code>fabs(x)</code>	计算 x 的绝对值
<code>log(x)</code>	计算 $\ln x$ 的值, x 应大于 0
<code>log10(x)</code>	计算 $\lg x$ 的值, x 应大于 0

函 数 名	功 能
<code>exp(x)</code>	计算 e^x 的值
<code>pow(x,y)</code>	计算 x^y 的值
<code>sin(x)</code>	计算 $\sin x$ 的值, x 为弧度值, 而非角度值
<code>cos(x)</code>	计算 $\cos x$ 的值, x 为弧度值, 而非角度值

如何进行更复杂的数学运算呢?



第三讲 学习内容—数据类型、表达式、程序控制流程

44

- 3.1 数据类型
 - 3.1.1 数据与变量
 - 3.1.2 基本数据类型
 - 3.1.3 复合与抽象类型
- 3.2 表达式与运算
 - 3.2.1 算术运算与表达式
 - 3.2.2 类型转换
 - 3.2.3 常用数学函数
- 3.3 程序控制流程
 - 3.3.1 顺序与选择结构
 - 3.3.2 循环结构
 - 3.3.3 流程控制语句



3.3.1 顺序与选择结构-什么是顺序结构

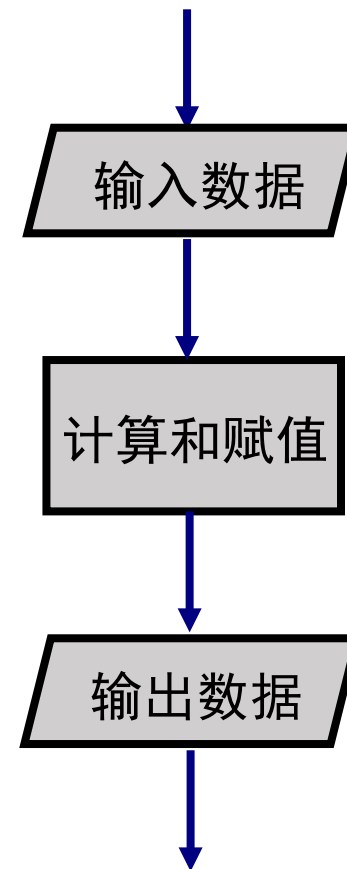
45

- 计算和赋值
 - 赋值表达式语句

赋值表达式 ;
`price = quantity * pricePerkg;`

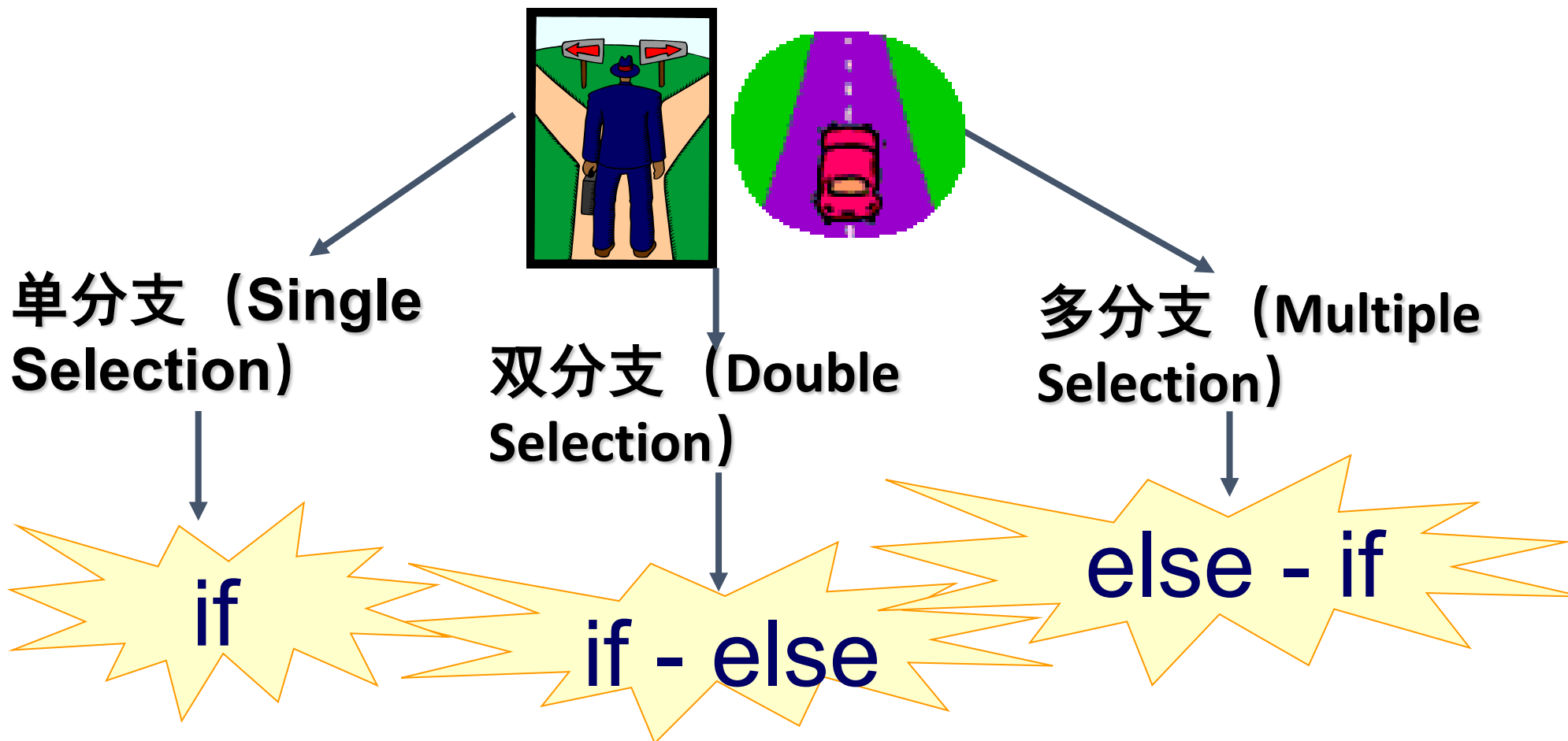
- 数据的输入输出
 - 标准输入输出函数调用语句

`scanf("%d", &pricePerkg);`
`printf("%d", price);`



3.3.1 顺序与选择结构-什么是选择结构

46



3.3.1 顺序与选择结构-用于单分支控制的条件语句

47

Syntax:

if (表达式)
语句;

or

if (表达式)
{
语句1;
语句2;
}

用**表达式**表示**条件**

不局限于**关系表达式** (0或1)

也可为**数值表达式** (0或非0)

表达式的值**非0**时, 为**真**

复合语句

(Compound Statement)

被当作一条语句看待

3.3.1 顺序与选择结构-用于双分支控制的条件语句

48

Syntax:

```
if ( 表达式 )  
    语句1;  
else  
    语句2;
```

即使分支中只有一条语句，大括号最好也不省略

or

```
if ( 表达式 )  
{  
    语句1;  
    语句3;  
}  
else  
{  
    语句2;  
    语句4;  
}
```


3.3.1 顺序与选择结构-条件运算符和条件表达式

49

```
#include <stdio.h>
main()
{
    int a, b, max;

    printf("Input a, b:");
    scanf("%d,%d", &a, &b);

    if (a > b)
        max = a;
    else
        max = b;

    printf("max = %d", max);
}
```

表达式1 ? 表达式2 : 表达式3

max = a > b ? a : b;

3.3.1 顺序与选择结构-用于多分支控制的条件语句

50

级联式if语句: else-if

Step a

if (表达式1)

{

Step m

}

else if(表达式2)

{

Step n

}

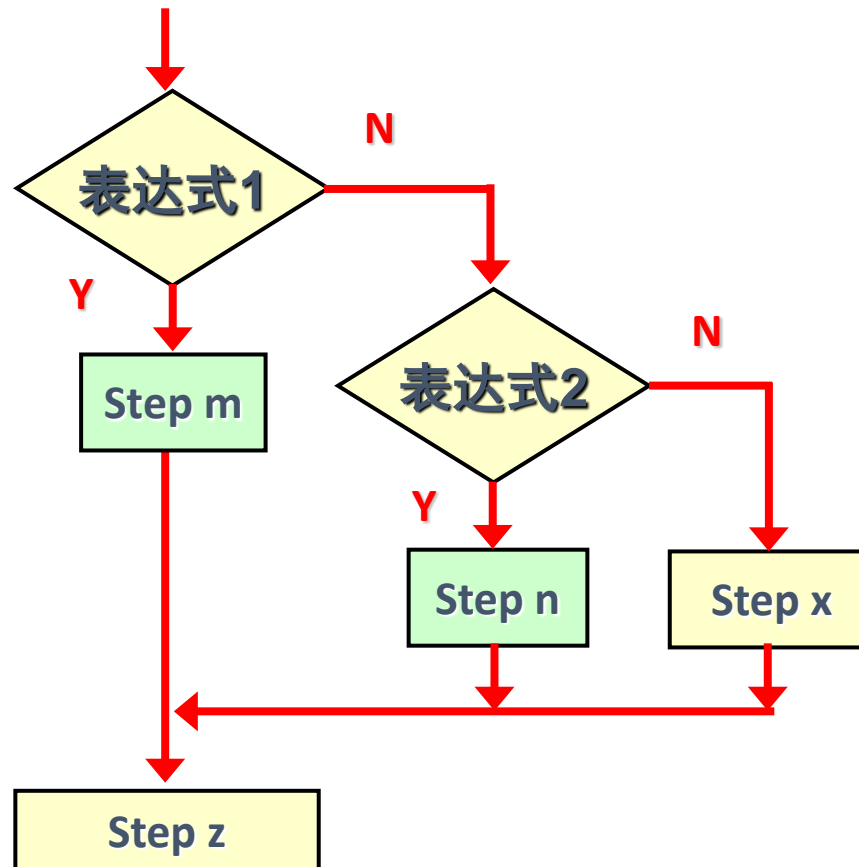
else

{

Step x

}

Step z



3.3.1 顺序与选择结构-用于多路选择的switch语句

51

开关语句

```
switch (表达式)
{
    case value1:
        语句1;
        break;
    case value2:
        语句2;
        break;
    .....
    default:
        语句n;
        break;
}
```

必须是int或char!

必须是常量!!

它是如何执行的
的呢?

必须有空格!!



第三讲 学习内容—数据类型、表达式、程序控制流程

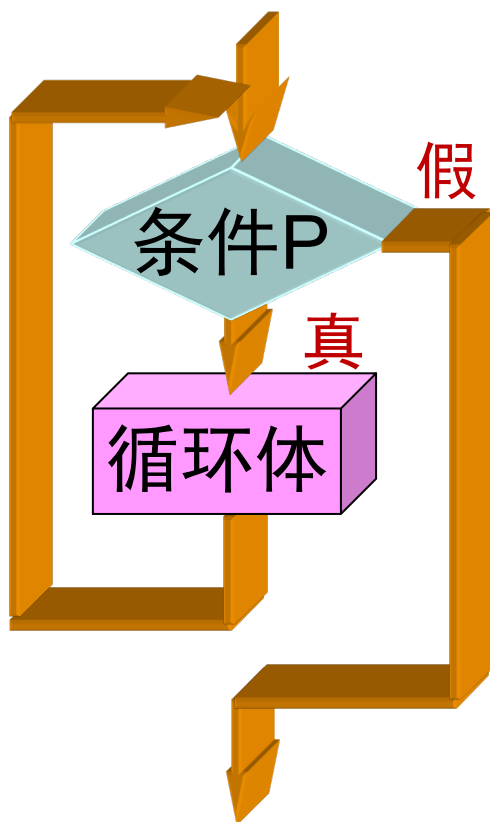
52

- 3.1 数据类型
 - 3.1.1 数据与变量
 - 3.1.2 基本数据类型
 - 3.1.3 复合与抽象类型
- 3.2 表达式与运算
 - 3.2.1 算术运算与表达式
 - 3.2.2 类型转换
 - 3.2.3 常用数学函数
- 3.3 程序控制流程
 - 3.3.1 顺序与选择结构
 - 3.3.2 循环结构
 - 3.3.3 流程控制语句



3.3.2 循环结构-循环结构类型

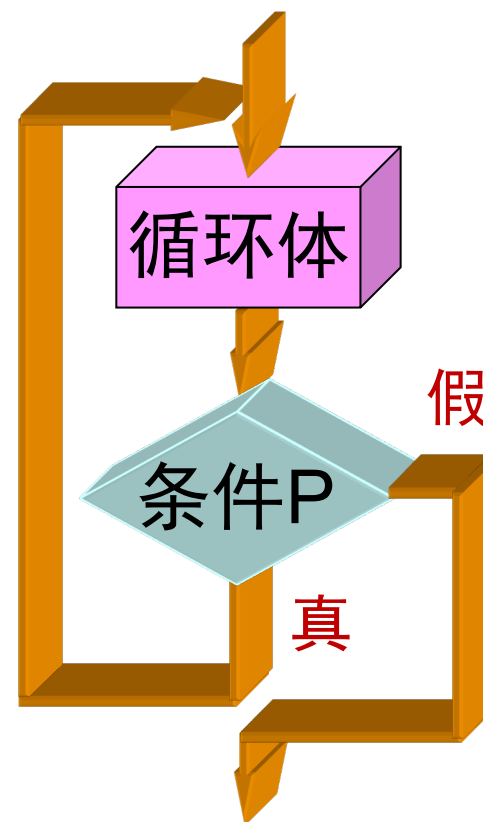
53



当型循环

Condition is tested **first**

VS

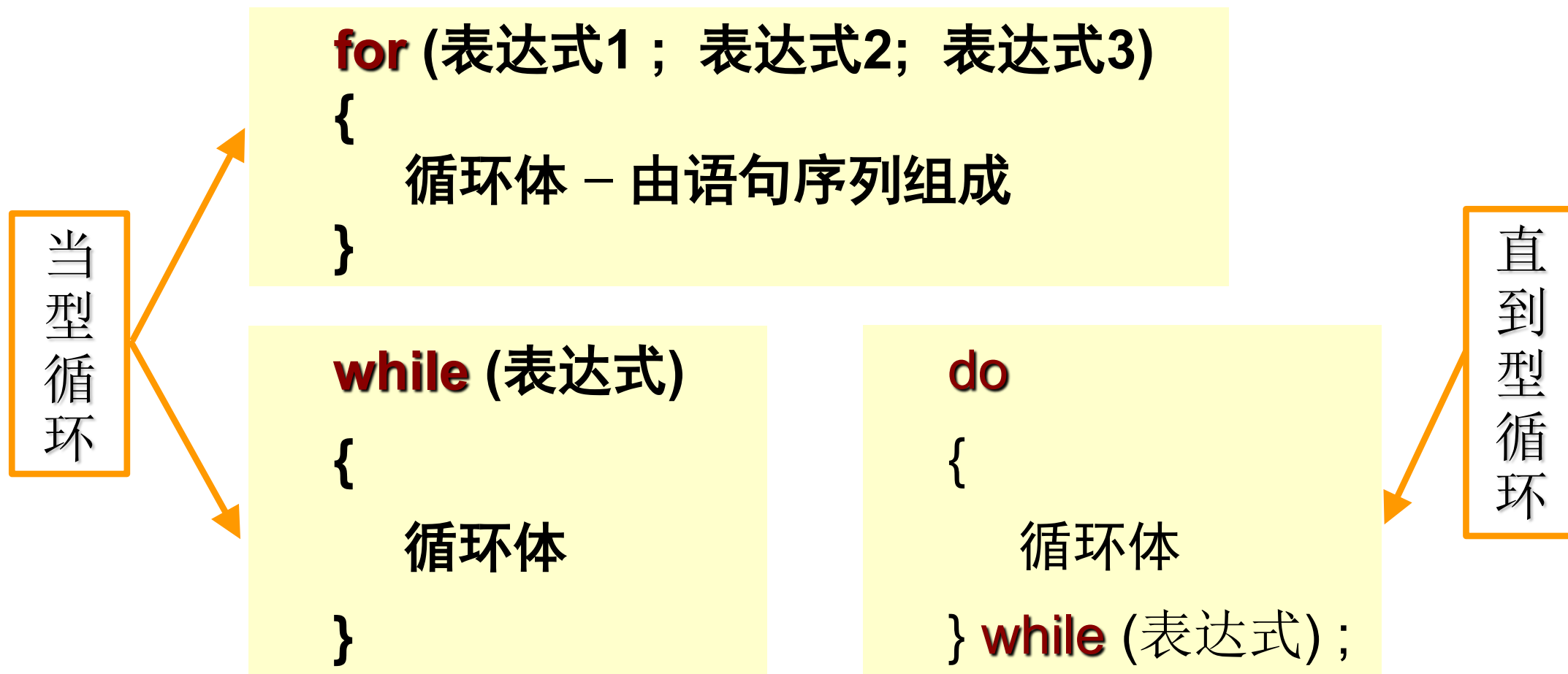


直到型循环

Condition is tested **last**

3.3.2 循环结构-循环语句

54



3.3.2 循环结构-for循环语句

55

- 计数控制——Loop is controlled by a counter

循环初始条件

循环控制条件

循环转化条件

- Syntax

for (表达式1 ;
 语句

Or

表达式2;

表达式3)

控制循环次数

不要重复在循环体内改
变循环变量的值

for (表达式1 ;

表达式2;

表达式3)

{

语句1

语句2

}

复合语句 (Compound
Statement) 做循环体
被当作一条语句看待

3.3.2 循环结构-while循环语句

56

- 当型循环——Condition is tested **first**

- **Syntax**

语句0

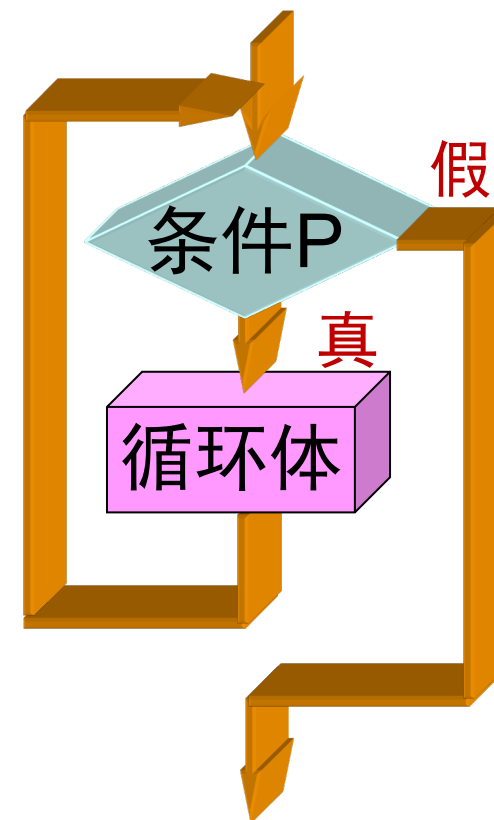
while (表达式)

```
{  
    语句1  
    语句2  
}
```

循环初始条件

循环控制条件

循环转化条件



3.3.2 循环结构- do-while循环语句

57

- 直到型循环——condition is tested **last**

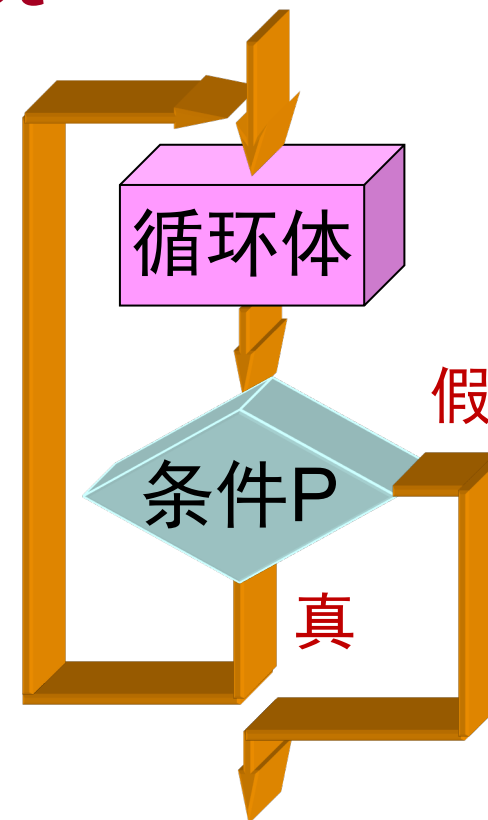
- **Syntax**

```
语句0  
do{  
    语句1  
    语句2  
}while (表达式);
```

循环初始条件

循环转化条件

循环控制条件



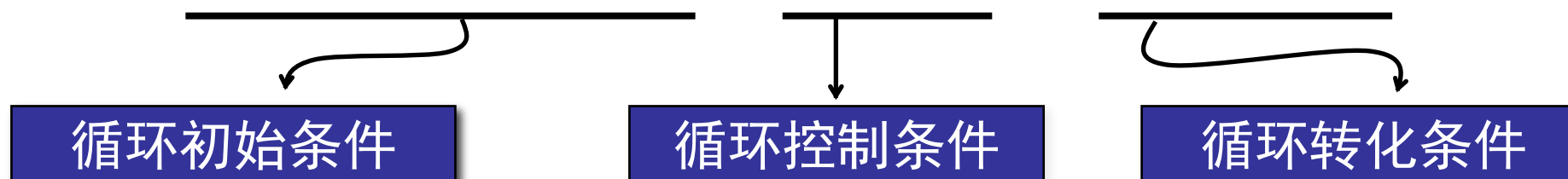
3.3.2 循环结构-逗号运算符

58

表达式1, 表达式2, ..., 表达式n

- 返回最后一个表达式的值，但一般不用整个表达式的值
- 更常见的是分别得到各表达式的值——顺序求值运算符
- 主要用在循环语句中，同时对多个变量赋初值等

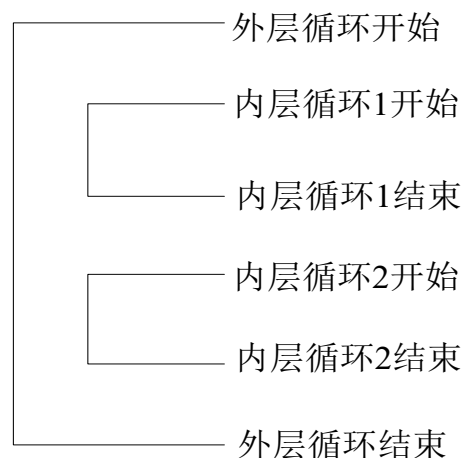
`for (i = 1 , j = n; i <= j; i++, j--)`



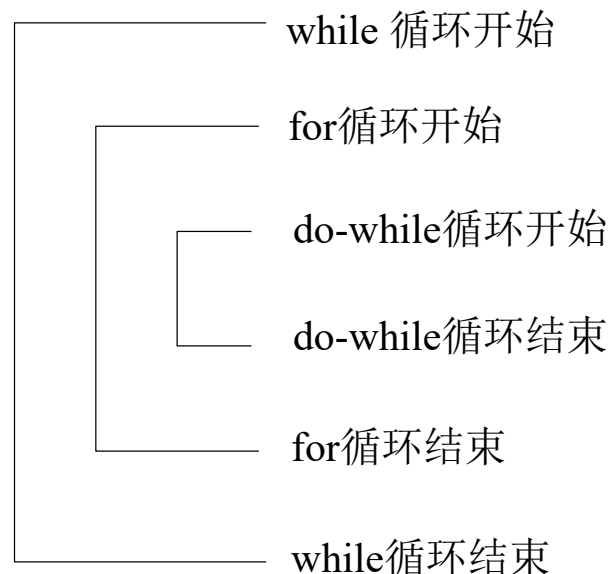
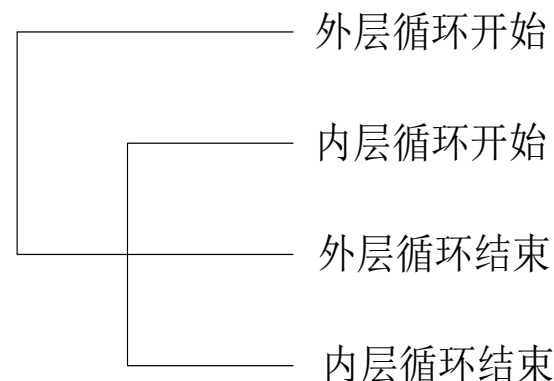
3.3.2 循环结构-嵌套循环(多重循环)

59

正确嵌套结构



错误嵌套结构



■ 嵌套循环注意事项

- 内层和外层循环控制变量不能同名，以免造成混乱；
- 用一对花括号将各层循环体语句括起来；
- 采用右缩进格式书写，以保证层次的清晰性

第三讲 学习内容—数据类型、表达式、程序控制流程

60

- 3.1 数据类型
 - 3.1.1 数据与变量
 - 3.1.2 基本数据类型
 - 3.1.3 复合与抽象类型
- 3.2 表达式与运算
 - 3.2.1 算术运算与表达式
 - 3.2.2 类型转换
 - 3.2.3 常用数学函数
- 3.3 程序控制流程
 - 3.3.1 顺序与选择结构
 - 3.3.2 循环结构
 - 3.3.3 流程控制语句



3.3.3 流程控制语句 - **break**语句 和 **continue**语句

61

- **break**语句 和 **continue**语句
 - 对for、while、do-while循环进行内部手术

